

Bases de Datos — 2. Curso

Grado en Ingenieria Matematica e Inteligencia Artificial (IMAT)

Reviews Amazon

Analisis de mas de 474.000 resenas de productos Amazon
con MySQL, MongoDB y Neo4J

Alumna	Claudia Maria Lopez Bombin
Asignatura	Bases de Datos
Curso	2. Grado IMAT
Tecnologias	MySQL 8 – MongoDB 7 – Neo4J 5 – Python 3.11
Total datos	474.344 reviews en 4 categorias

Contents

1	Introduccion	3
2	Primera parte: Diseno y carga de datos	3
2.1	Distribucion de datos: MySQL vs. MongoDB	3
2.2	Esquema relacional MySQL	4
2.3	Proceso de carga — load_data.py	4
3	Segunda parte: Menu de visualizacion	5
3.1	Opcion 1 — Evolucion de reviews por anos	5
3.2	Opcion 2 — Popularidad de articulos	6
3.3	Opcion 3 — Histograma por nota	7
3.4	Opcion 4 — Evolucion acumulada de reviews	7
3.5	Opcion 5 — Distribucion de reviews por usuario	8
3.6	Opcion 6 — Nubes de palabras (MongoDB)	9
3.7	Opcion 7 — Ratio de utilidad por categoria (extra)	10
4	Tercera parte: Grafos en Neo4J	10
4.1	Apartado 4.1 — Similitudes de Pearson entre usuarios	10
4.2	Apartado 4.2 — Articulos aleatorios y usuarios	11
4.3	Apartado 4.3 — Usuarios multi-categoria	11
4.4	Apartado 4.4 — Articulos populares con menos de 40 resenas	12
5	Cuarta parte: Nuevo dataset	12
6	Quinta parte: Recomendaciones y Machine Learning	12
6.1	Top 10 articulos no consumidos — recomendaciones.py	12
6.2	Modelo de Machine Learning propuesto	13
7	Estructura del proyecto y division del trabajo	13

1 Introduccion

El proyecto aborda el diseno, implementacion y analisis de una base de datos hibrida para gestionar resenas de productos Amazon. Los datos provienen de cuatro datasets en formato JSON:

Table 1: Datasets utilizados

Dataset	Fichero	Reviews
Toys and Games	Toys_and_Games_5.json	167.597
Video Games	Video_Games_5.json	231.780
Digital Music	Digital_Music_5.json	64.706
Musical Instruments	Musical_Instruments_5.json	10.261
Total		474.344

Cada linea de los ficheros JSON tiene el siguiente formato:

```

1 {
2   "reviewerID":      "A2HD75EMZR8QLN",  # ID del autor
3   "asin":           "0700099867",      # ID del articulo
4   "reviewerName":    "Maria",
5   "helpful":         [8, 12],           # [votos utiles,
        total votos]
6   "reviewText":      "Texto largo...",
7   "overall":         5.0,               # Nota (1.0 a 5.0)
8   "summary":         "Resumen breve",
9   "unixReviewTime":  1341792000,
10  "reviewTime":      "07 9, 2012"       # formato MM DD,
        YYYY
11 }
```

Listing 1: Formato de una review en los ficheros JSON

2 Primera parte: Diseno y carga de datos

2.1 Distribucion de datos: MySQL vs. MongoDB

Datos discretos y estructurados, optimos para JOIN eficientes y agregaciones rapidas.

Columnas TEXT enormes en SQL penalizarian las consultas de agregacion. MongoDB gestiona texto variable eficientemente.

2.2 Esquema relacional MySQL

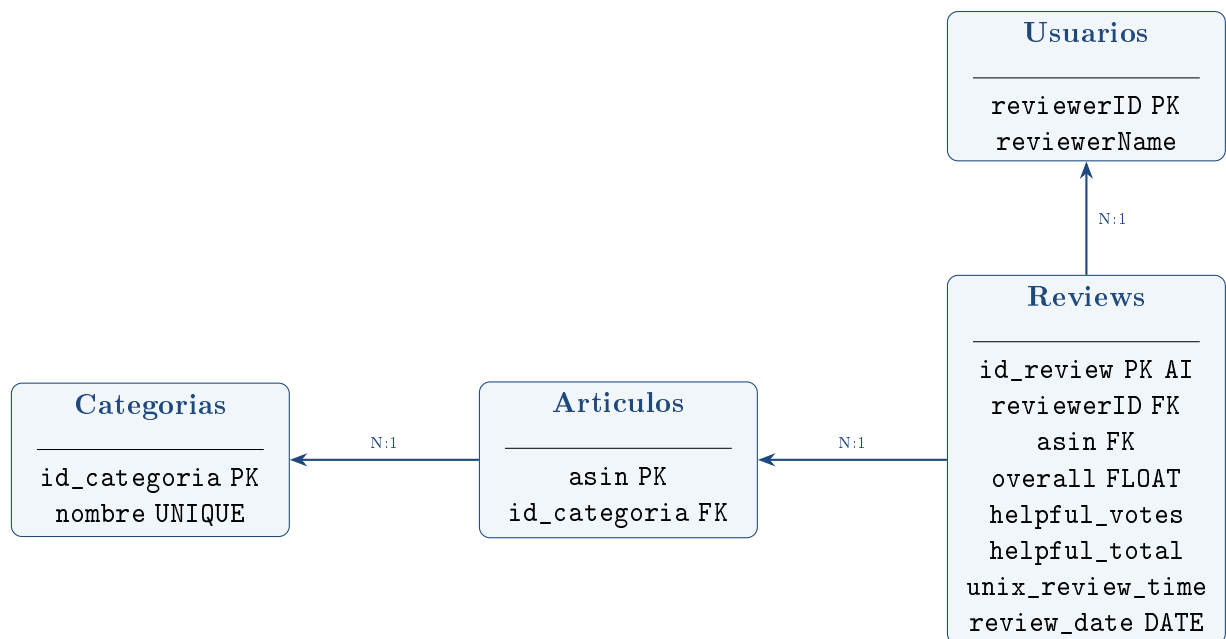


Figure 1: Esquema relacional de la base de datos MySQL

```

1 CREATE TABLE Reviews (
2     id_review          INT          NOT NULL AUTO_INCREMENT,
3     reviewerID         VARCHAR(50) NOT NULL,
4     asin               VARCHAR(20) NOT NULL,
5     overall            FLOAT        NOT NULL,
6     helpful_votes      INT          DEFAULT 0,
7     helpful_total      INT          DEFAULT 0,
8     unix_review_time   BIGINT,
9     review_date        DATE,
10    PRIMARY KEY (id_review),
11    FOREIGN KEY (reviewerID) REFERENCES Usuarios(reviewerID)
12    ,
13    FOREIGN KEY (asin) REFERENCES Articulos(asin),
14    INDEX idx_fecha     (review_date),
15    INDEX idx_usuario   (reviewerID),
16    INDEX idx_articulo  (asin)
17 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
  
```

Listing 2: DDL de la tabla Reviews (fragmento de load_data.py)

2.3 Proceso de carga — load_data.py

La carga cumple todos los requisitos del enunciado: lectura línea a línea (`for línea in f`), sin pandas, sin LOAD DATA, con inserción en batch de 500 registros mediante `cursor.executemany()` y cache en set para evitar duplicados.

```

1 with open(ruta_json, "r", encoding="utf-8") as f:
2     for línea in f:
3         # nunca se
4         carga el fichero entero
  
```

```

3      review = json.loads(linea.strip())
4      fecha_sql = parsear_fecha(review.get("reviewTime", ""))
5
6      if reviewer_id not in cache_usuarios:  # cache en memoria
7          cursor.execute("INSERT IGNORE INTO Usuarios ..."
8                          , ...)
9          cache_usuarios.add(reviewer_id)
10
11     batch_mysql.append((reviewer_id, asin, overall, ...))
12
13     batch_mongo.append({"summary": summary, ...})
14
15     if len(batch_mysql) >= 500:  # insercion en bloque
16         cursor.executemany("INSERT INTO Reviews ...",
17                             batch_mysql)
18         conexion.commit()
19         coleccion.insert_many(batch_mongo)
20         batch_mysql, batch_mongo = [], []

```

Listing 3: Lectura linea a linea e insercion en batch

3 Segunda parte: Menu de visualizacion

El fichero `menu_visualizacion.py` implementa un menu interactivo por terminal que se repite en bucle hasta la opcion 0. Todas las graficas se guardan automaticamente en `outputs/graficas/` con nombre descriptivo y timestamp.

3.1 Opcion 1 — Evolucion de reviews por anos

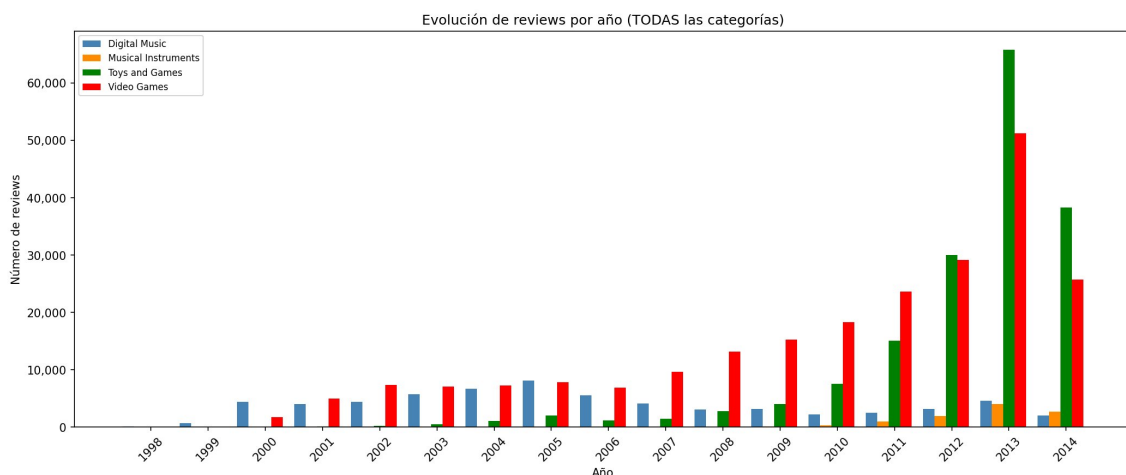


Figure 2: Evolucion de reviews por ano — todas las categorías (barras agrupadas)

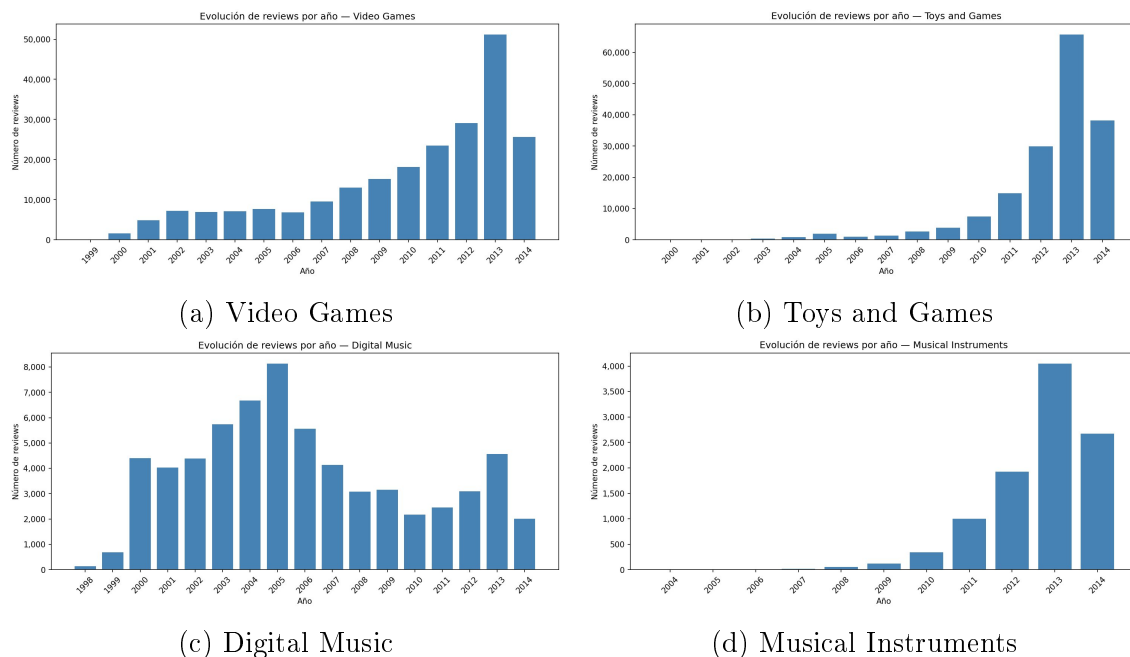


Figure 3: Evolucion por año desglosada por categoria

Se observa un crecimiento exponencial en Video Games y Toys and Games, con pico en 2013. Digital Music tiene su maximo en 2005 (auge del iPod) y decrece, lo que refleja el cambio hacia el streaming. Musical Instruments muestra un arranque tardio pero acelerado desde 2011.

3.2 Opcion 2 — Popularidad de articulos

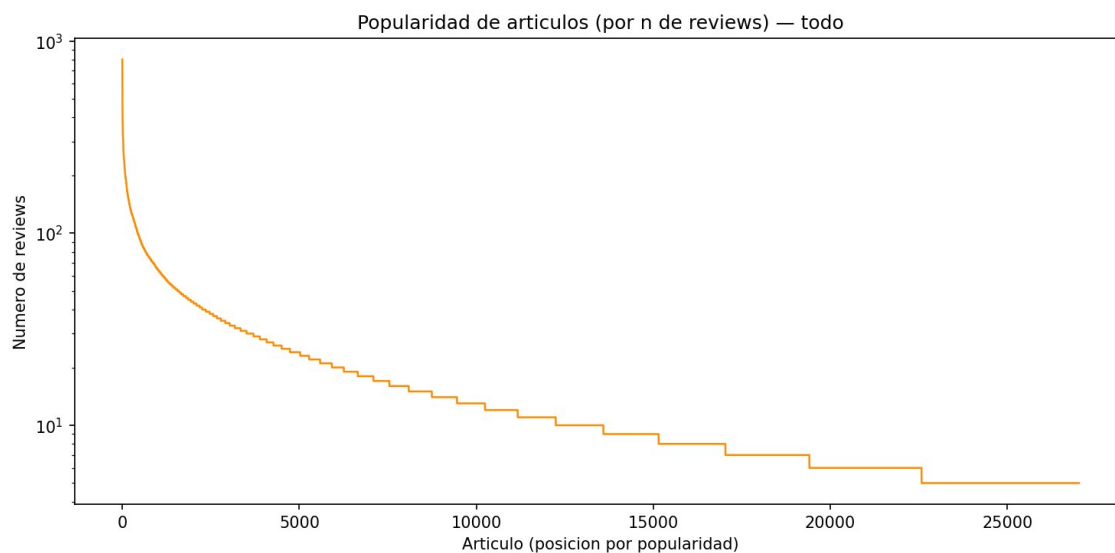


Figure 4: Curva de popularidad de articulos en escala logaritmica

La curva confirma una distribucion de ley de potencias (cola larga): el articulo mas popular supera las 900 resenas mientras que la mayoria de articulos tiene entre 5 y 10. Esto es consistente con la Ley de Pareto en comercio electronico.

3.3 Opcion 3 — Histograma por nota

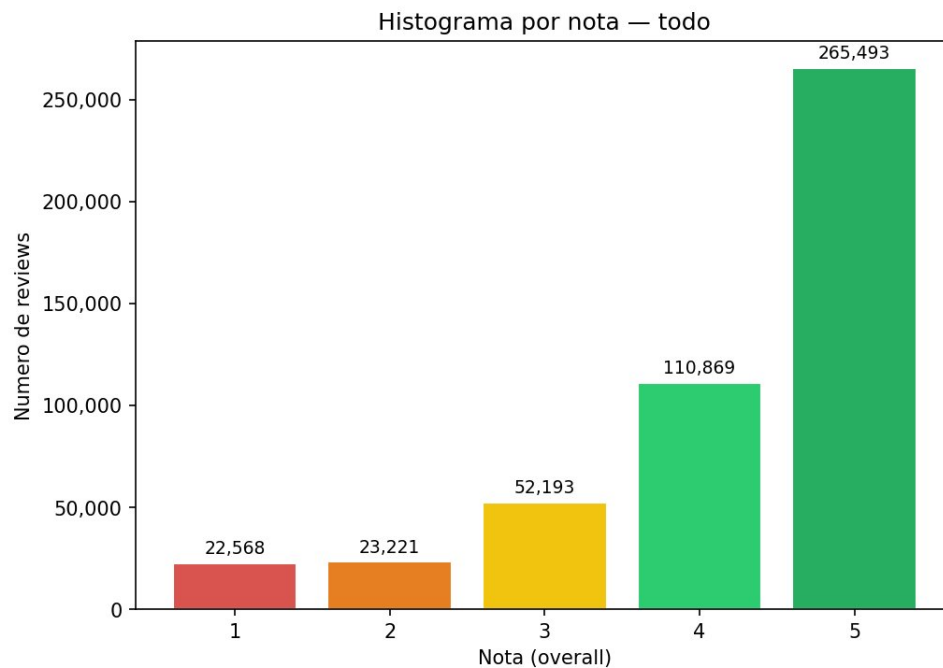


Figure 5: Distribucion de notas overall — todas las categorias

La distribucion es claramente asimetrica: el 55,9% de las resenas tienen nota 5 (265.493 resenas). Las notas 1 y 2 representan menos del 10% del total. Este fenomeno conocido como J-curve refleja que los usuarios satisfechos son mas propensos a dejar una resena.

3.4 Opcion 4 — Evolucion acumulada de reviews

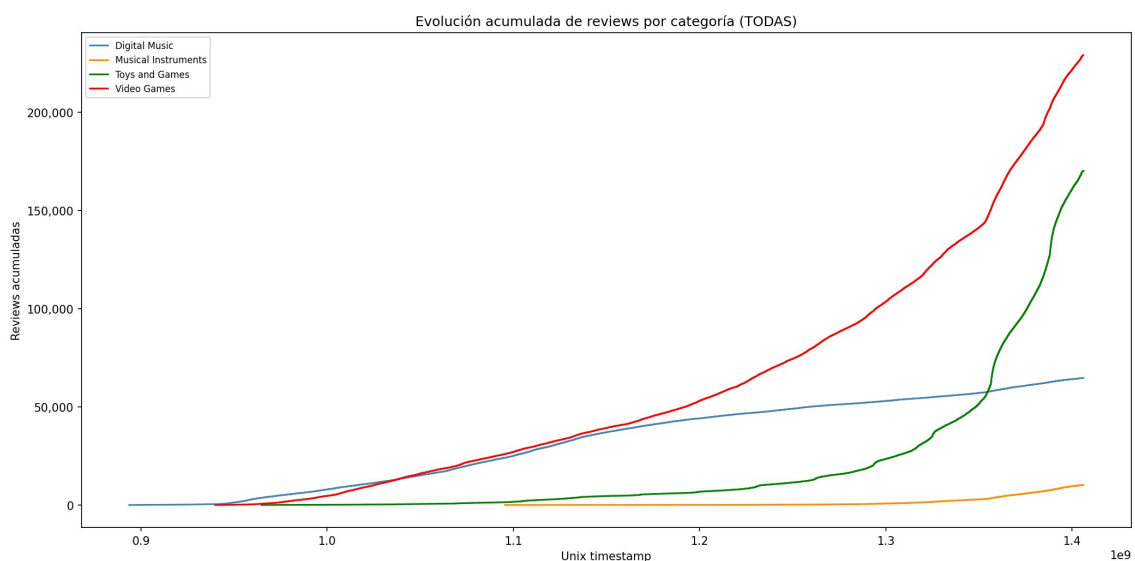


Figure 6: Evolucion acumulada por categorias (unix timestamp en eje X)

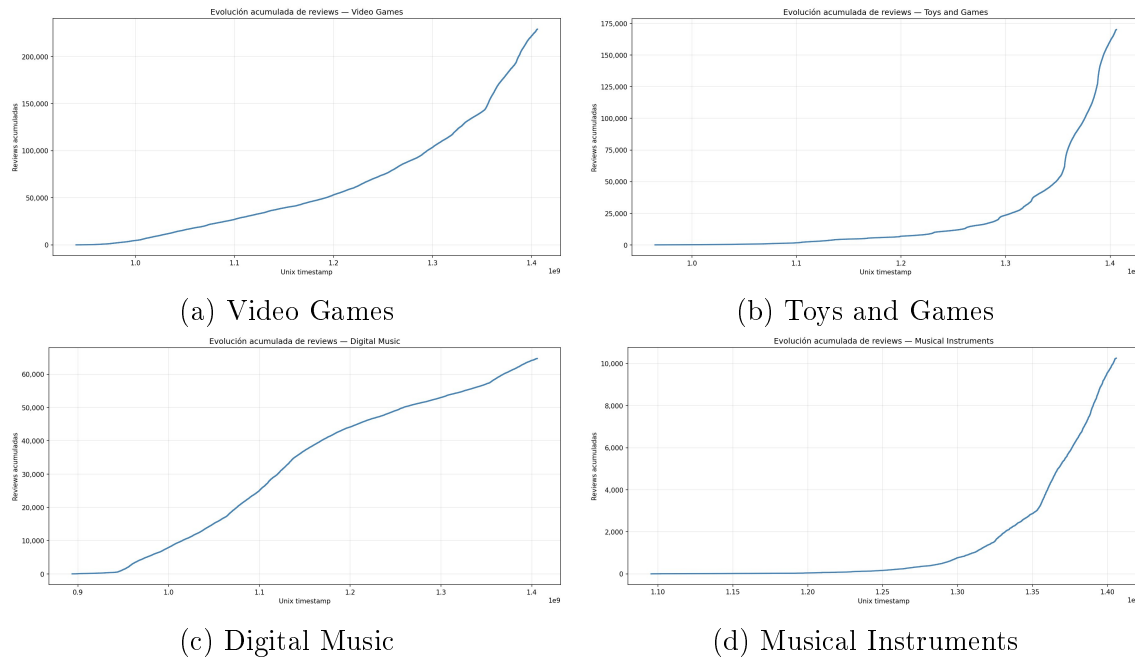


Figure 7: Evolucion acumulada desglosada por categoria

Video Games y Toys and Games muestran aceleracion exponencial a partir de 2012 ($\approx 1.35 \times 10^9$ en timestamp unix). Musical Instruments tiene crecimiento casi nulo hasta 2011 y luego sube abruptamente. Digital Music muestra crecimiento lineal sostenido.

3.5 Opcion 5 — Distribucion de reviews por usuario

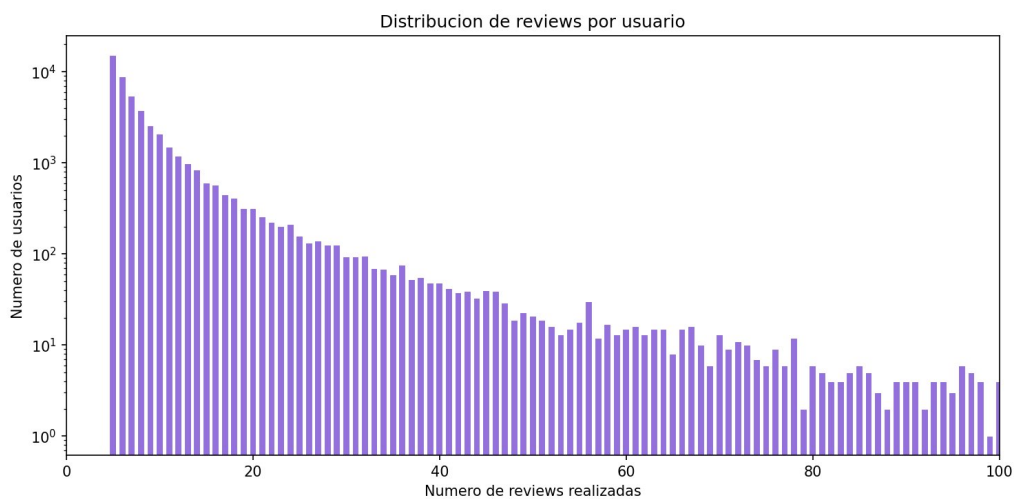


Figure 8: Histograma logaritmico de reviews por usuario

La distribucion sigue una ley de potencias: la mayoría de usuarios ha hecho entre 5 y 10 resenas, mientras que unos pocos super-reviewers superan las 80-100. La escala logaritmica en el eje Y es imprescindible para visualizar correctamente esta distribucion.

3.7 Opcion 7 — Ratio de utilidad por categoria (extra)

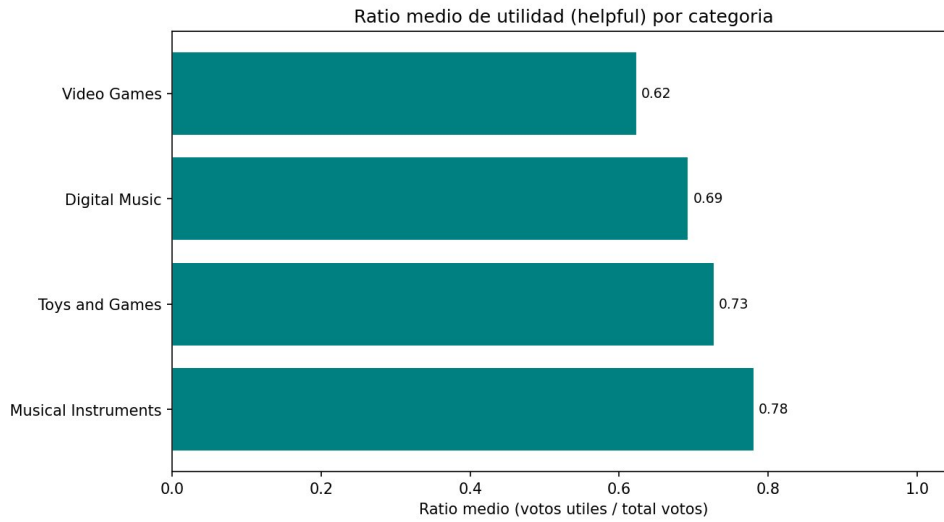


Figure 10: Ratio medio de utilidad helpful por categoria

Musical Instruments tiene el ratio mas alto (0.78): sus resenas son percibidas como mas utiles porque los compradores buscan informacion tecnica especifica. Video Games tiene el ratio mas bajo (0.62), posiblemente porque las resenas son mas subjetivas y polarizadas.

4 Tercera parte: Grafos en Neo4J

El fichero `neo4JProyecto.py` implementa un menu con 4 opciones. Cada opcion limpia la base de datos Neo4J antes de cargar sus datos.

4.1 Apartado 4.1 — Similitudes de Pearson entre usuarios

Se obtienen los $N = 30$ usuarios con mas resenas (configurable con `TOP_USUARIOS_SIMILITUD`). Para cada par (u, v) :

$$PC(u, v) = \frac{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)^2} \cdot \sqrt{\sum_{i \in I_{uv}} (r_{vi} - \bar{r}_v)^2}} \quad (1)$$

Solo se almacenan pares con $|I_{uv}| > 0$. Las similitudes se guardan en `similitudes_cache.json` y se cargan en Neo4J como relaciones `SIMILAR` con propiedad `similitud`. La consulta para el usuario con mas vecinos es:

```
1 MATCH (u:Usuario)-[:SIMILAR]-(vecino)
2 RETURN u.reviewerID AS usuario, COUNT(vecino) AS num_vecinos
3 ORDER BY num_vecinos DESC LIMIT 1;
```

Listing 4: Cypher — usuario con mas vecinos

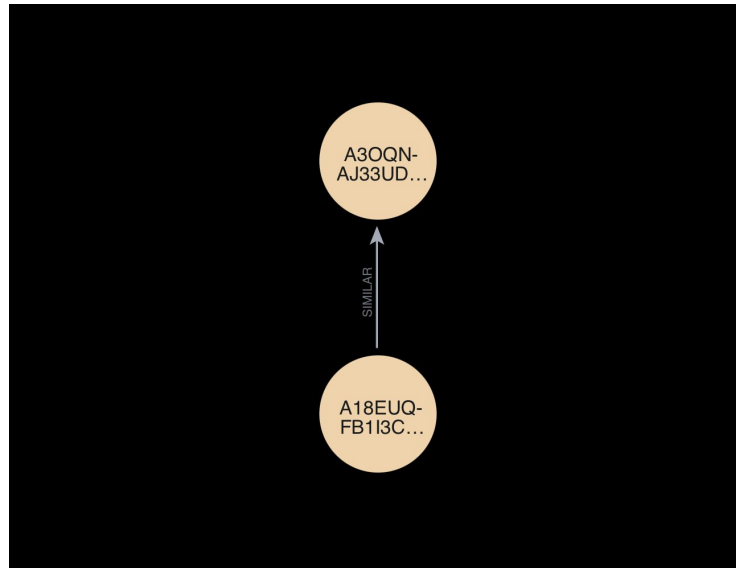


Figure 11: Grafo de similitudes Pearson entre usuarios en Neo4J Browser (apartado 4.1)

4.2 Apartado 4.2 — Artículos aleatorios y usuarios

El usuario introduce la categoría y el número n . Se seleccionan con `random.sample()` y se cargan nodos `:Articulo` y `:Usuario` con enlace `VALORO` (propiedades: `nota` y `timestamp`).

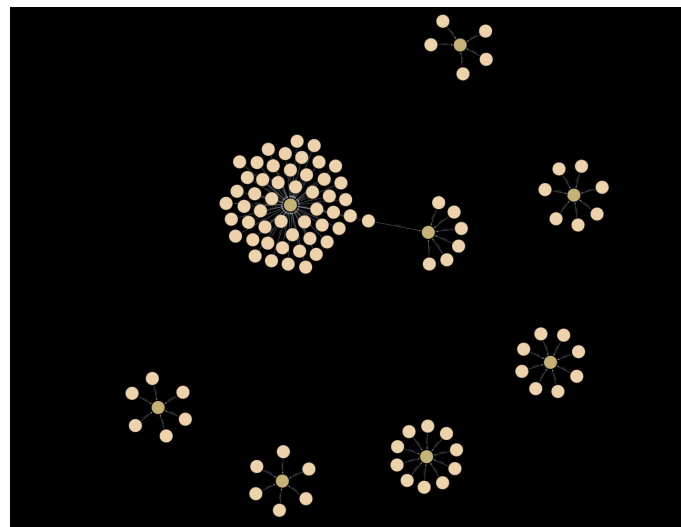


Figure 12: Grafo usuario-articulo con relacion VALORO en Neo4J Browser (apartado 4.2)

4.3 Apartado 4.3 — Usuarios multi-categoria

De los primeros 400 usuarios por nombre, se filtran los que han valorado ≥ 2 tipos distintos de artículos. Grafo: `:Usuario` `—[CONSUME]—>` `:Categoria` con propiedad `num_articulos`.

4.4 Apartado 4.4 — Artículos populares con menos de 40 reseñas

```
1 SELECT asin, COUNT(*) AS total_reviews
2 FROM Reviews
3 GROUP BY asin
4 HAVING total_reviews < 40
5 ORDER BY total_reviews DESC
6 LIMIT 5;
```

Listing 5: SQL — 5 artículos mas populares con menos de 40 reseñas

Se cargan sus usuarios y se crean relaciones EN_COMUN entre pares de usuarios con propiedad `articulos_comunes`.

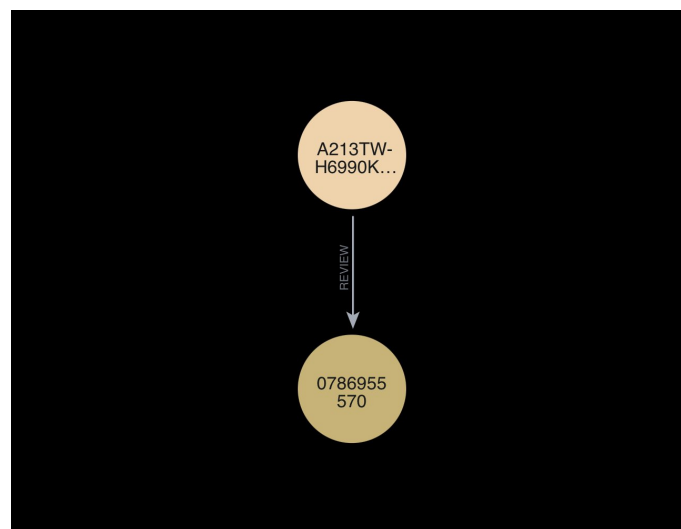


Figure 13: Grafo artículos populares y relaciones EN_COMUN entre usuarios (apartado 4.4)

5 Cuarta parte: Nuevo dataset

`inserta_dataset.py` es un CLI que reutiliza la infraestructura de `load_data.py`:

```
python inserta_dataset.py --ruta datos/Sports_and_Outdoors_5.json
                           --categoria "Sports and Outdoors"
```

Carga previamente los `reviewerID` ya existentes en un `set` para no duplicar usuarios que ya tienen reseñas en los 4 datasets originales.

6 Quinta parte: Recomendaciones y Machine Learning

6.1 Top 10 artículos no consumidos — `recomendaciones.py`

Dados un `reviewerID` y una categoría, devuelve los 10 artículos mas populares de esa categoría que el usuario aun no ha valorado. Los artículos ya consumidos se cargan en un `set` Python y se filtran antes de devolver el resultado.

6.2 Modelo de Machine Learning propuesto

Se propone un modelo de Filtrado Colaborativo basado en usuarios (User-Based CF):

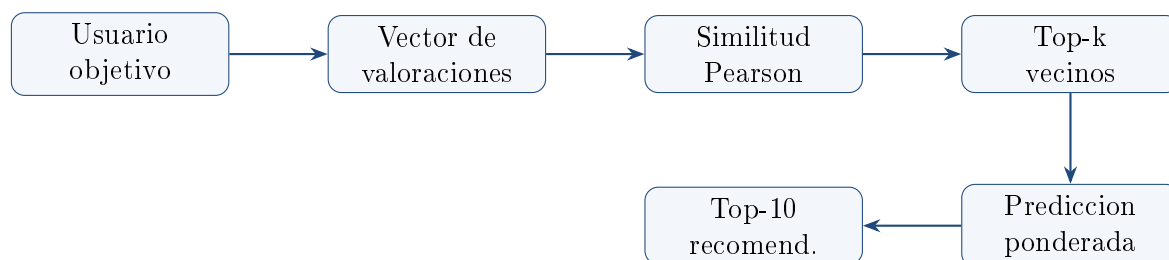


Figure 14: Diagrama del modelo de Machine Learning propuesto

La valoracion predicha del articulo i para el usuario u es:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N(u)} PC(u, v) \cdot (r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |PC(u, v)|}$$

Como alternativa escalable: Matrix Factorization con SVD (libreria `surprise`) o ALS implícito (libreria `implicit`).

7 Estructura del proyecto y division del trabajo

Table 2: Ficheros entregados

Fichero	Descripcion	LOC
<code>configuracion.py</code>	Credenciales, rutas y parametros	~50
<code>load_data.py</code>	Schema MySQL/MongoDB y carga linea a linea	~220
<code>menu_visualizacion.py</code>	Menu de 7 visualizaciones + guardado auto	~250
<code>neo4JProyecto.py</code>	Menu Neo4J con 4 apartados	~280
<code>inserta_dataset.py</code>	CLI para insertar nuevos datasets	~80
<code>recomendaciones.py</code>	Top 10 articulos no consumidos	~100
<code>requirements.txt</code>	Dependencias Python	—
<code>README.md</code>	Instrucciones de ejecucion paso a paso	—
Total lineas de codigo		~980

El proyecto ha sido realizado integramente de forma **individual** por Claudia Maria Lopez Bombin. Todas las partes han sido desarrolladas por la misma alumna.

Claudia Maria Lopez Bombin — Proyecto Final Bases de Datos — IMAT 2. Curso